

SPENGERGASSE  ausbildung mit zukunft	HTBLVA, Wien V, Spengergasse Aufbaulehrgang für Informatik Kolleg für Informatik	Reife- und Diplomprüfung
--	---	-------------------------------------

Aufgabenstellung für die nicht standardisierte Klausurprüfung

Herbsttermin 2025/26

Jahrgang:	6AAIF, 6BAIF, 6CAIF, 6AKIF, 6BKIF, 8ABIF, 8ACIF
Prüfungsgebiet:	Fachtheorie
Zugeteilter Pflichtgegenstand (Zugeteilte Pflichtgegenstände)	Programmieren und Software Engineering
Prüfungstag:	21. September 2026
Arbeitszeit:	300 Minuten
Kandidaten/Kandidatinnen:	60
Prüfer/Prüferin:	Michael Schletz, BEd und René Wenz, BSc
Aufgabenblätter:	11 Blätter plus Umschlagbogen

(Unterschrift des Prüfers/der Prüferin auf den jeweiligen Aufgabenblättern)

Das versiegelte Kuvert mit der Aufgabenstellung wurde geöffnet von:

Name: _____ Unterschrift: _____

Datum: _____ Uhrzeit: _____

Zwei Zeugen (Kandidaten/Kandidatinnen)

Name: _____ Unterschrift: _____

Name: _____ Unterschrift: _____

Geprüft: Wien, am _____

 AV Mag. Heidi Steinwender
 Abteilungsvorständin

RS.

 Dir. Dr. Gerhard Hager
 Direktor

Genehmigt:

Wien, am _____

RS.

 Mag.^a MinRⁱⁿ Gabriele Winkler-Rigler
 Schulaufsicht

Klausurprüfung in Programmieren und Software Engineering

Klassen: 6xAIF, 6xKIF, 8ABIF, 8ACIF

Datum: MO, 21. September 2026

Arbeitszeit: 300 Minuten

Generelle Hinweise zur Bearbeitung

Sie haben 5 Stunden (300 Minuten) Zeit für alle Aufgaben. Die 3 Teilaufgaben sind unabhängig voneinander. Sie können die Reihenfolge und die Zeit frei wählen. Wir empfehlen: höchstens 2 Stunden für Aufgabe 1, 1,5 Stunden für Aufgabe 2 und 1,5 Stunden für Aufgabe 3. Damit die Prüfung in die Jahresnote zählt, brauchen Sie mindestens 30 % der Gesamtpunkte.

Hilfsmittel

Auf dem Netzlaufwerk **P:/SPG_Fachtheorie** finden Sie ein fertiges Projekt für Ihren Technologiestack. Alle Pakete (Dependencies) sind schon installiert.

Im Labor steht Visual Studio mit .NET 10 oder IntelliJ Ultimate mit mindestens JDK 21 bereit. Das Projekt liegt auf dem Netzlaufwerk. Sie müssen Ihre Lösung am Ende also nicht extra kopieren. **Schließen** Sie am Ende **alle Programme** und lassen Sie den PC eingeschaltet. Erlaubte Hilfsmittel finden Sie im Ordner **R:\exams**.

Wichtiger Hinweis vor Arbeitsbeginn



Tragen Sie Ihre Daten (Klasse, Name und Accountname) in die Datei *README.md* unter *SPG_Fachtheorie/README.md* ein. Sie sehen die Datei nach dem Öffnen der Solution unter *Solution Items*. **Ohne diese Angaben können wir Ihre Arbeit nicht zuordnen und nicht bewerten!**



Während der Prüfung gibt es keinen Internetzugang. Visual Studio: Verwenden Sie **nie Build - Rebuild Solution**. Das löscht die lokalen Pakete. Verwenden Sie immer *Build - Build Solution* (F6).

Teilaufgabe 1: Object Relation Mapping

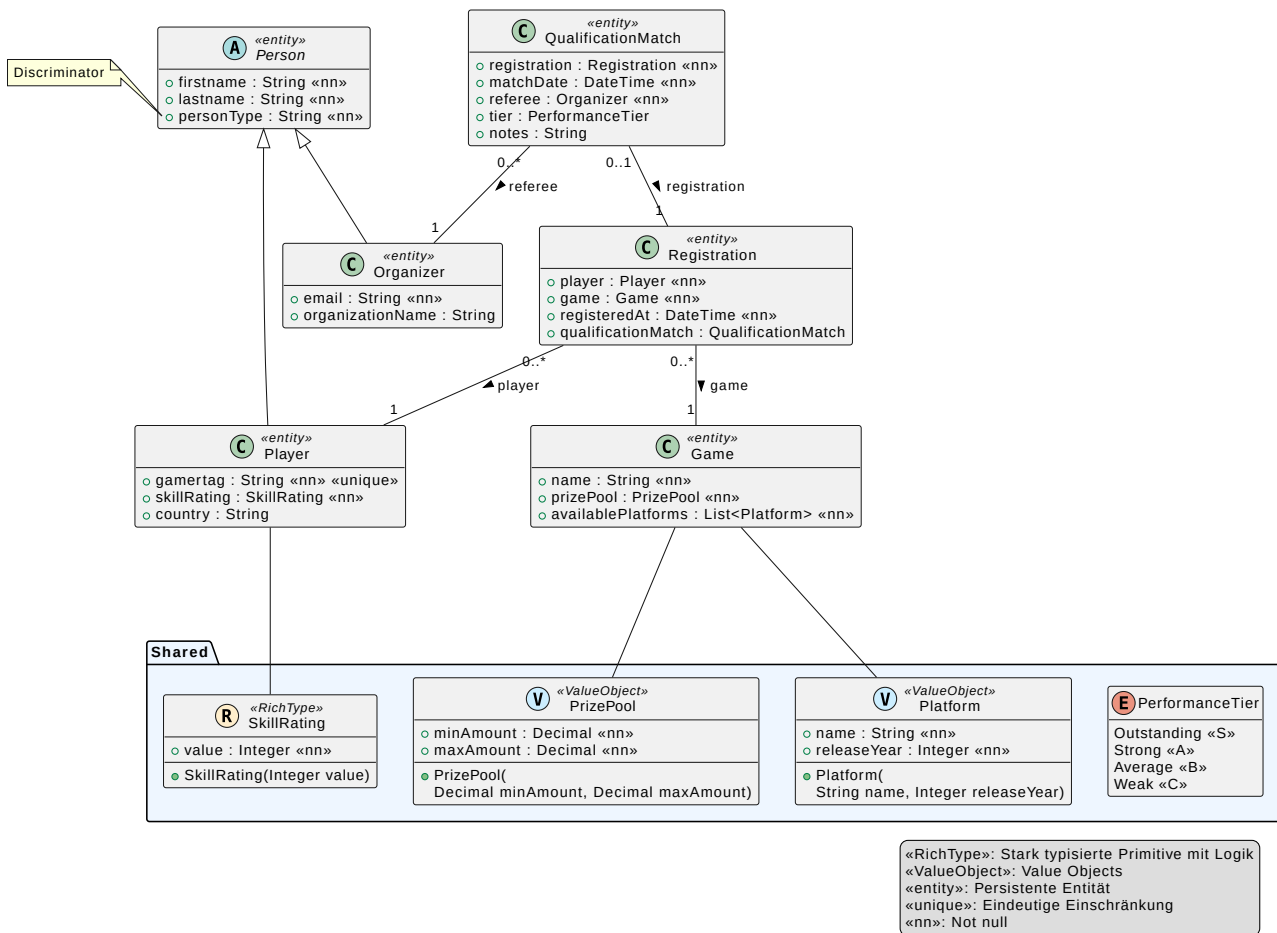
Das Domänenmodell ist technologie-neutral beschrieben. Es nutzt UML-Namensregeln: Felder beginnen mit einem kleinen Buchstaben und die Datentypen sind neutral. Die neutralen Datentypen entsprechen diesen Typen:

UML (neutral)	.NET (C#)	Java
Integer	int	Integer / int
String	string	String
Decimal	decimal	BigDecimal
Boolean	bool	Boolean / boolean
Date	DateOnly	LocalDate
DateTime	DateTime	LocalDateTime

In .NET sind die *nullable reference types* aktiv. Felder ohne `<<nn>>` (not null) sind daher *nullable* (zum Beispiel `string?` in C# oder ein optionales Feld in Java).

Eine Plattform verwaltet Esports-Wettbewerbe. Das System speichert die Spielerinnen und Spieler (*Player*). Sie nehmen an Wettbewerben teil. Es speichert auch die Veranstalterinnen und Veranstalter (*Organizer*). Sie organisieren und leiten die Wettbewerbe. Ein *Player* kann sich direkt für ein Spiel (*Game*, zum Beispiel "Counter-Strike 2") anmelden. So entsteht eine *Registration* (Anmeldung). Zu einer Anmeldung kann ein *Organizer* ein *QualificationMatch* (Qualifikationsspiel) durchführen und bewerten.

So sieht das Domain Model aus:



Arbeitsauftrag

Erstellung der Modelklassen

Setzen Sie das Diagramm als Modelklassen für den OR Mapper um.

Im Projekt *Aufgabe1* finden Sie leere Klassen und die Klasse *RegistrationContext* (C#). Nutzen Sie diese.

Beachten Sie dabei folgende Punkte:

Allgemeine Anforderungen

- Legen Sie die nötigen Konstruktoren an. Ein *public* Konstruktor soll alle Properties aus dem Modell setzen. Fügen Sie bei Bedarf weitere Konstruktoren hinzu, die der OR Mapper braucht.
- Beachten Sie Regeln wie *not null* (<<nn>>).
- Nutzen Sie eigene primary keys mit dem Namen *Id* (autoincrement). Eine Ausnahme gilt nur, wenn im Modell mit <<PK>> ein eigener Schlüssel (PK) angegeben ist.
- Der Name eines Foreign Keys ist: Property-Name + Name des PK. Das ist zum Beispiel wich-

tig, wenn Sie den FK einer dependent Entity selbst setzen.

Fachliche Anforderungen

- Die Tabellennamen sollen in der Einzahl entstehen (*Person*, *Game*, *Registration*, ...).
- Zu einer *Registration* gibt es höchstens ein *QualificationMatch*. Setzen Sie diese 1:0..1 Beziehung um.
- Das Attribut *Player.gamertag* braucht ein *unique constraint*.
- Es darf keine zwei *Registrations* mit denselben Werten in *Player* und *Game* geben. Stellen Sie das mit einem *unique constraint* über diese 2 Spalten sicher (also über die beiden Fremdschlüssel).
- Definieren Sie das Attribut *Game.prizePool* als *value object*. Die Spalten sollen in der Datenbank *PrizePool_MinAmount* und *PrizePool_MaxAmount* heißen (das ist das Standardverhalten in EF Core).
- Definieren Sie das Attribut *Game.availablePlatforms* als Liste von *value objects*. Sorgen Sie durch die Konfiguration dafür, dass der primary key der neuen Tabelle *Id* heißt.
- Definieren Sie das Attribut *Player.skillRating* als *rich type* und speichern Sie es als *int*.
- Definieren Sie das Attribut *Person.personType* als discriminator.
- Speichern Sie die Enum *QualificationMatch.tier* als String mit diesen Werten: *S* für Outstanding, *A* für Strong, *B* für Average, *C* für Weak. Der Wert kann auch *null* sein.
- Prüfen Sie in der Klasse *SkillRating* den Wert im Konstruktor. Liegt der Wert nicht zwischen 0 und 5000 (inklusive), werfen Sie eine *ArgumentException*.
- Prüfen Sie in der Klasse *PrizePool* den Wert im Konstruktor. Ist *minAmount* größer als *maxAmount*, werfen Sie eine *ArgumentException*.



Wenn rich types oder enums Probleme machen, können Sie sie auch als einfaches String-Feld umsetzen.

Vorgegebene Tests

Der vorgegebene Test *T00_CanCreateDatabaseTest* in *Aufgabe1.Test/Aufgabe1MasterTests* prüft, ob Ihre Implementierung überhaupt eine Datenbank erzeugen kann. **Das ist die Grundvoraussetzung für Punkte in Aufgabe 1.**

Der vorgegebene Test *T00_SchemaTest* in *Aufgabe1.Test/Aufgabe1MasterTests* prüft das ganze Modell. Er schickt dafür *INSERT*-Befehle für alle Tabellen an die Datenbank ("happy path").

Verfassen von Tests

Schreiben Sie in der Klasse *RegistrationContextTests* (Projekt *Aufgabe1.Test*) Testmethoden. Diese Tests sollen zeigen, dass Sie den OR Mapper richtig konfiguriert haben.

- *T01_InsertRegistrationWithQualificationMatchTest* zeigt: Sie können eine *Registration* für ein

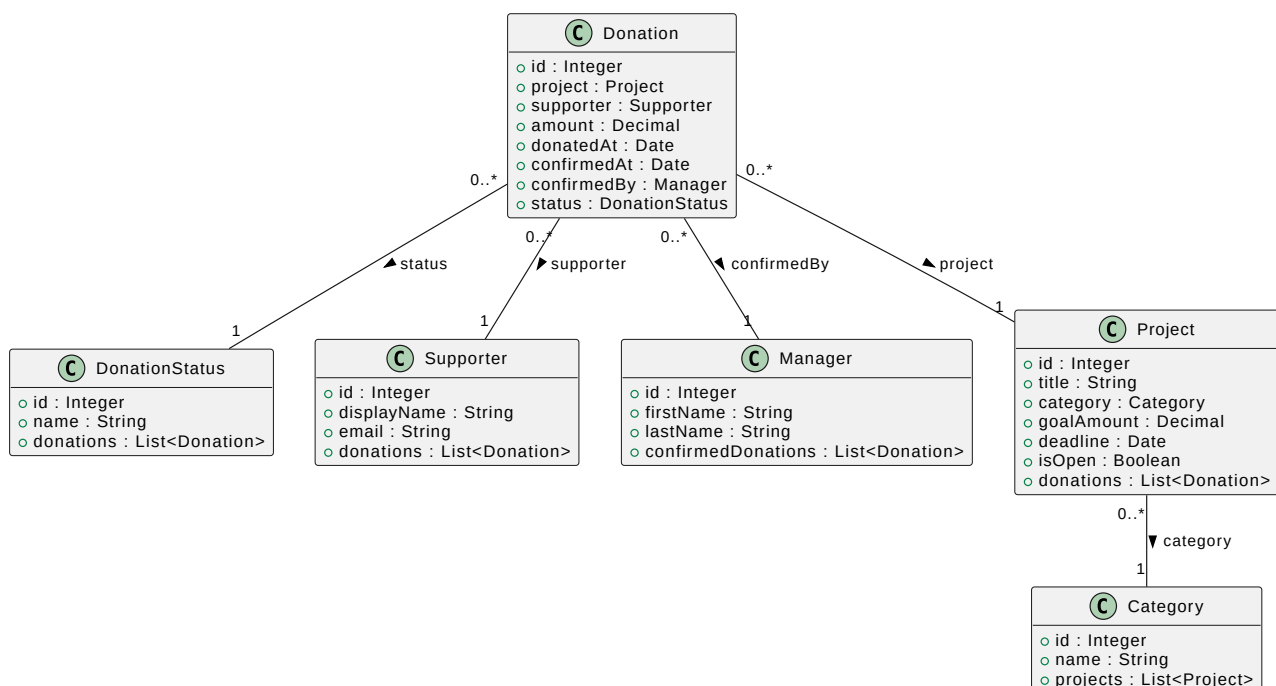
Game zusammen mit einem *QualificationMatch* speichern.

- *T02_RegistrationPlayerAndGameIsUniqueTest* zeigt: Eine zweite *Registration* mit demselben *Player* und demselben *Game* führt zu einer *DbUpdateException*.
- *T03_AddGameWithPlatforms* zeigt: Sie können ein *Game* mit mehreren *Platforms* speichern. Achten Sie im Assert darauf: Sie laden das *Game* ohne *Include* und greifen trotzdem auf die Liste der *Platforms* zu (prüfen Sie die Anzahl).
- *T04_PrizePoolThrowsExceptionWhenMinMaxIsInvalidTest* zeigt: Der Konstruktor von *PrizePool* wirft bei ungültigen Argumenten eine *ArgumentException*.

Teilaufgabe 2: Abfragen und Servicemethoden

Das folgende Modell beschreibt eine einfache Crowdfunding-Plattform. Ein *Supporter* ist eine Person, die Projekte unterstützt. Ein *Supporter* kann Geld für ein *Project* spenden. Jede Spende ist ein Datensatz in *Donation*. Ein *Manager* ist ein/e MitarbeiterIn. Sobald ein *Manager* eine Spende bestätigt, werden *confirmedAt* und *confirmedBy* gesetzt. Vorher sind diese Werte *null*. Jedes *Project* gehört zu einer Kategorie (*Category*). Es hat ein Spendenziel (*goalAmount*) und ein Flag *isOpen*. Das Flag zeigt, ob das Projekt noch Spenden annimmt. Der Status ist eine Lookup-Tabelle mit drei Werten: *Pending* (offen, noch nicht bestätigt), *Confirmed* (bestätigt) und *Refunded* (zurückgezahlt).

Das Domain Model ist technologieneutral beschrieben (UML-Namensregeln).



Musterdaten

Im Ordner *datafiles* liegen Musterdaten im TSV-Format. Sie können diese Textdateien in Ihrer

IDE ansehen.

Arbeitsauftrag

In der Datei *BaseModel/Infrastructure/DonationContext.cs* ist ein fertiger DbContext vorhanden (C#). In Java finden Sie fertige Repositories und Fixtures. Das Modell ist bereits in *BaseModel/Model* umgesetzt. **Sie müssen das Modell nicht selbst implementieren. Sie verwenden es nur in Ihren Servicemethoden.** Schreiben Sie Ihren Code in die Klasse *Aufgabe2/Donation-Service*. Verwenden Sie in C# den *DonationContext*, den Sie über Dependency Injection bekommen bzw. die vorgegebenen Repositories in Java.



Für alle Auswertungen zählen nur **bestätigte** Spenden, also Donations mit dem Statusnamen *"Confirmed"*. Achten Sie bei Vergleichen auf die korrekte Großschreibung. Spenden mit Status *Pending* oder *Refunded* zählen nicht.

Implementierung der Servicemethoden

Setzen Sie die folgenden Abfragen und Servicemethoden um:

List<CategoryFunding> GetCategoryFunding()

Geben Sie für jede Kategorie (*Category*) zwei Werte zurück: die Summe der bestätigten Spendenbeträge und die Anzahl der Projekte in dieser Kategorie. Sortieren Sie das Ergebnis **absteigend** (descending) nach der Spendensumme. Die Kategorie mit dem meisten Geld steht oben. Nutzen Sie diesen Record als Rückgabebetyp.

```
// C#  
public record CategoryFunding(string Name, decimal TotalAmount, int ProjectCount);
```

```
// Java  
public record CategoryFunding(String name, BigDecimal totalAmount, int projectCount) {}
```



Sie summieren hier zweimal. Pro Projekt summieren Sie *Donation.amount* über alle bestätigten Donations. Danach summieren Sie diese Projekt-Summen über alle Projekte der Kategorie.

List<Supporter> GetSupportersForCategory(string category)

Geben Sie alle *Supporter* zurück, die **mindestens eine** bestätigte Spende für ein Projekt der gesuchten Kategorie gemacht haben. Vergleichen Sie dafür *Category.name* mit dem Parameter *category*.

List<ProjectProgress> GetUnderfundedProjects(decimal maxPercent)

Geben Sie alle noch offenen Projekte (*isOpen* ist *true*) zurück. Wählen Sie nur Projekte aus, deren bestätigte Spendensumme **weniger als** *maxPercent* Prozent vom Spendenziel (*goalAmount*) erreicht hat. Geben Sie pro Projekt auch den Betrag zurück, der noch bis zum Ziel fehlt. Nutzen Sie diesen Record als Rückgabetyt.

```
// C#
public record ProjectProgress(
    string Title, decimal GoalAmount,
    decimal CollectedAmount, decimal MissingAmount);
```

```
// Java
public record ProjectProgress(
    String title, BigDecimal goalAmount,
    BigDecimal collectedAmount, BigDecimal missingAmount) {}
```



Ein Projekt ist *underfunded*, wenn gilt: $\text{collected} < \text{goalAmount} * \text{maxPercent} / 100$. *collected* ist die Summe der bestätigten Donations. Achten Sie bei der Division auf den Datentyp (decimal)!

Donation AddDonation(int supporterId, int projectId, decimal amount, DateOnly donatedAt)

Diese Methode legt eine Spende (*Donation*) an. Prüfen Sie vorher diese Bedingungen:

- Gibt es keinen *Supporter* mit der *supporterId* in der Datenbank, werfen Sie eine *DonationException* mit dem Text "Invalid supporter."
- Gibt es kein *Project* mit der *projectId* in der Datenbank, werfen Sie eine *DonationException* mit dem Text "Invalid project."
- Ist *Project.isOpen* gleich *false*, werfen Sie eine *DonationException* mit dem Text "Project is closed."

Sind alle Bedingungen erfüllt, legen Sie ein neues *Donation*-Objekt an und speichern Sie es in der Datenbank. Verwenden Sie den Status *Confirmed*. Setzen Sie *confirmedAt* auf den Wert von *donatedAt*. *confirmedBy* bleibt *null*.

Danach aktualisieren Sie den Status des Projekts: Berechnen Sie die Summe der bestätigten Spenden dieses Projekts inklusive der neuen Spende. Erreicht oder überschreitet diese Summe das Spendenziel (*goalAmount*), schließen Sie das Projekt. Setzen Sie dafür *Project.isOpen* auf *false* und speichern Sie das Projekt.

Testen Ihrer Implementierung

In *Aufgabe2.Test/Aufgabe2MasterTests* gibt es schon fertige Tests für Ihre Implementierung. Sie müssen hier keine eigenen Unittests schreiben.

Teilaufgabe 3a: Erstellen einer RESTful API

Im Projekt *Aufgabe3* gibt es die Klasse *ProjectsController*. Diese Klasse bekommt über Dependency Injection ein fertiges Service (*ProjectService*). **Verwenden Sie dieses fertige Service. Sie brauchen Ihr Service aus Aufgabe 2 nicht.**

Das Service hat 3 Methoden bzw. Properties:

IQueryable<Project> Projects

Liefert alle *Project*-Datensätze aus der Datenbank. Sie brauchen das für die GET-Methode im Controller.

Task<Project> AddProject(AddProjectCmd cmd)

Legt aus dem Command ein neues *Project* an und gibt es zurück. Wirft eine *ProjectServiceException*, wenn das Speichern nicht möglich ist (zum Beispiel bei einer *CategoryId*, die es nicht gibt).

Task UpdateStatus(UpdateProjectStatusCmd cmd)

Setzt das Flag *isOpen* eines Projekts neu, basierend auf dem Command. Wirft eine *ProjectNotFoundException*, wenn es das *project* nicht gibt. Wirft eine *ProjectServiceException*, wenn das Speichern nicht möglich ist.

Zu implementierende Endpunkte

GET /projects?withDonations=(true|false)

Diese Route gibt eine Liste aller Projekte (*Project*) zurück. Ist der Query-Parameter *withDonations* gleich *true*, geben Sie pro Projekt auch die Liste seiner Spenden zurück. Ist *withDonations* gleich *false*, geben Sie bei *donations* ein leeres Array (*[]*) zurück. Nutzen Sie diesen Record für die Rückgabe:

```
// C#
public record ProjectWithDonationsDto(
    int Id, string Title, decimal GoalAmount,
    List<DonationDto> Donations);

public record DonationDto(
    int Id, DateOnly DonatedAt, decimal Amount,
    string SupporterDisplayName);
```

```
// Java
public record ProjectWithDonationsDto(
    int id, String title, BigDecimal goalAmount,
    List<DonationDto> donations) {}

public record DonationDto(
    int id, LocalDate donatedAt, BigDecimal amount,
    String supporterDisplayName) {}
```

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Ein Array von <i>ProjectWithDonationsDto</i> . Es enthält alle Projekte und – je nach Parameter – die Spenden.

GET /projects/{id}

Diese Route gibt das *Project* mit der übergebenen ID zurück.

Nutzen Sie diesen Record für die Rückgabe:

```
// C#
public record ProjectDto(int Id, string Title, decimal GoalAmount, bool IsOpen);
```

```
// Java
public record ProjectDto(int id, String title, BigDecimal goalAmount, boolean isOpen) {}
```

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Ein (1) <i>ProjectDto</i> Objekt mit den entsprechenden Daten.
404	Die ID wurde nicht gefunden.

POST /projects

Diese Route legt einen neuen *Project*-Datensatz an. Nutzen Sie dieses Command-Objekt:

```
// C#
public record AddProjectCmd(
    string Title, int CategoryId, decimal GoalAmount,
    decimal MinDonationAmount, DateOnly Deadline, bool IsOpen);
```

```
// Java
public record AddProjectCmd(
    String title, int categoryId, BigDecimal goalAmount,
    BigDecimal minDonationAmount, LocalDate deadline, boolean isOpen) {}
```

Beachten Sie bei der Validierung diese Regeln:

- *Title* ist ein String mit 1 bis 255 Zeichen.
- *CategoryId* ist mindestens 1.
- *GoalAmount* ist mindestens 1.
- *MinDonationAmount* ist mindestens 1.

Bauen Sie zusätzlich eine **feldübergreifende Validierung** ein. Nutzen Sie dafür *InvalidatableObject* (C#) bzw. eine klassenweite Validierung (Java). Es gilt diese Regel:

- *MinDonationAmount* darf nicht größer als *GoalAmount* sein. Eine einzelne Mindestspende kann nicht größer als das ganze Spendenziel sein (Fehlertext zum Beispiel *Minimum donation must not exceed the goal amount.*).

Bei einem Validierungsfehler oder einem Fehler der Business-Logik senden Sie HTTP 400 (bad request) mit einem RFC-Problem-Detail-Body.

War das Einfügen erfolgreich, senden Sie HTTP 201 Created. Senden Sie im Payload die ID des neuen *project*-Objekts im Property *id*. Senden Sie im Location-Header die Adresse der passenden GET-Methode (*GET /projects/{id}*).

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
201	Das project konnte angelegt werden.
400	Ein Validierungs- oder Servicefehler ist aufgetreten (z. B. eine nicht vorhandene <i>CategoryId</i> oder eine verletzte <i>InvalidatableObject</i> -Regel).

PATCH /projects/{id}/status

Diese Route aktualisiert das Feld *isOpen* eines *Project*. Die ID des *Project*-Datensatzes bekommen Sie über den Routing-Parameter. Nutzen Sie dieses Command-Objekt:

```
// C#
public record UpdateProjectStatusCmd(int Id, bool IsOpen);
```

```
// Java
public record UpdateProjectStatusCmd(int id, boolean isOpen) {}
```

Der Request-Body enthält das ganze Command-Objekt, also die Felder *id* und *isOpen*. Vergleichen Sie die ID aus dem Routing-Parameter mit dem Property *Id* aus dem Command-Objekt. Sind die beiden Werte verschieden, senden Sie HTTP 400 (bad request) mit dem Text *Invalid ID*. im RFC-Problem-Detail-Body.

Gibt es kein *Project* mit dieser ID, senden Sie HTTP 404 (not found) mit einem RFC-Problem-Detail-Body.

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
204	Das Flag <i>IsOpen</i> konnte aktualisiert werden.
400	Die ID aus dem Routingparameter und das Property <i>Id</i> im Command Object stimmen nicht überein.
404	Die ID wurde nicht gefunden.

HTTP-File zum Testen Ihrer Implementierung

Im Projekt *Aufgabe3* gibt es die Datei *Aufgabe3.http*. Damit können Sie Ihre Implementierung von Hand testen.

Teilaufgabe 3b: Erstellen von End-to-End (Integration) Tests (6 Punkte)

Schreiben Sie im Projekt *Aufgabe3.Test* in der Klasse *ProjectsControllerTests* Integration-Tests für den Endpunkt *POST /projects*. **Testen Sie keine anderen Endpunkte.**

Schreiben Sie Tests für diese zwei Szenarien:

- *POST /projects* legt mit gültigen Daten einen Datensatz an und liefert HTTP 201 Created. Prüfen Sie den Statuscode und prüfen Sie, ob der Datensatz in der Datenbank steht.
- *POST /projects* kann keinen Datensatz anlegen, weil es keine Kategorie mit dieser ID gibt. Prüfen Sie den Statuscode.